Azure Spray

Investigating a password spray attack against a Microsoft 365 tenant using Microsoft Sentinel and KQL

Tools	Microsoft Sentinel (Logs, Analytics, Incidents), Kusto Query Language (KQL), Microsoft Entra ID sign-in logs
Techniques	Log triage, statistical aggregation, attacker infrastructure profiling, detection engineering, incident review
MITRE ATT&CK	T1110.003 Brute Force: Password Spraying (Credential Access); T1078.004 Valid Accounts: Cloud Accounts

Overview

A mid-sized technology company recently migrated to Microsoft 365 and its identity logs began showing a large volume of failed sign-ins. The task was to determine whether the tenant was under a password spray attack, in which an adversary tries a small set of common passwords against many accounts so that no single account crosses a lockout threshold, and to establish the scope of the activity, find any account that was successfully breached, and turn the findings into an automated detection.

The investigation was carried out entirely in Microsoft Sentinel, using KQL against the custom Entra ID sign-in log table (`SigninLogs_CL`), and ended with a working scheduled analytics rule that raised an incident for the attack.

Objectives

- Triage the sign-in logs and isolate the failure pattern that characterises the attack
- Profile the attacker: source infrastructure, client fingerprint and geography
- Establish the attack timeline and the number of accounts targeted
- Build a Sentinel analytics rule from a Microsoft template, adapted to the custom log schema
- Review the resulting incident, identify the compromised account, and document mitigations

Investigation Flow

1. Log Triage	2. Source Profiling	3. Timeline and Scope	4. Detection Engineering	5. Incident Review	6. Compromise Identification	7. Mitigation

PHASE 1 Sign-in Log Triage

The first step was to understand what the sign-in table contained. Grouping every event by its `ResultType` and sorting by volume gives a quick picture of which outcomes dominate. In a healthy tenant successful sign-ins vastly outnumber failures; here a single failure code dwarfed everything else, which is the first hint of automated credential guessing. A password spray leaves a recognisable signature: a very large count of "invalid username or password" failures, with only a handful of lockout or other error codes appearing once defences start reacting.

```
SigninLogs_CL
| summarize Count = count() by ResultType
| order by Count desc
```

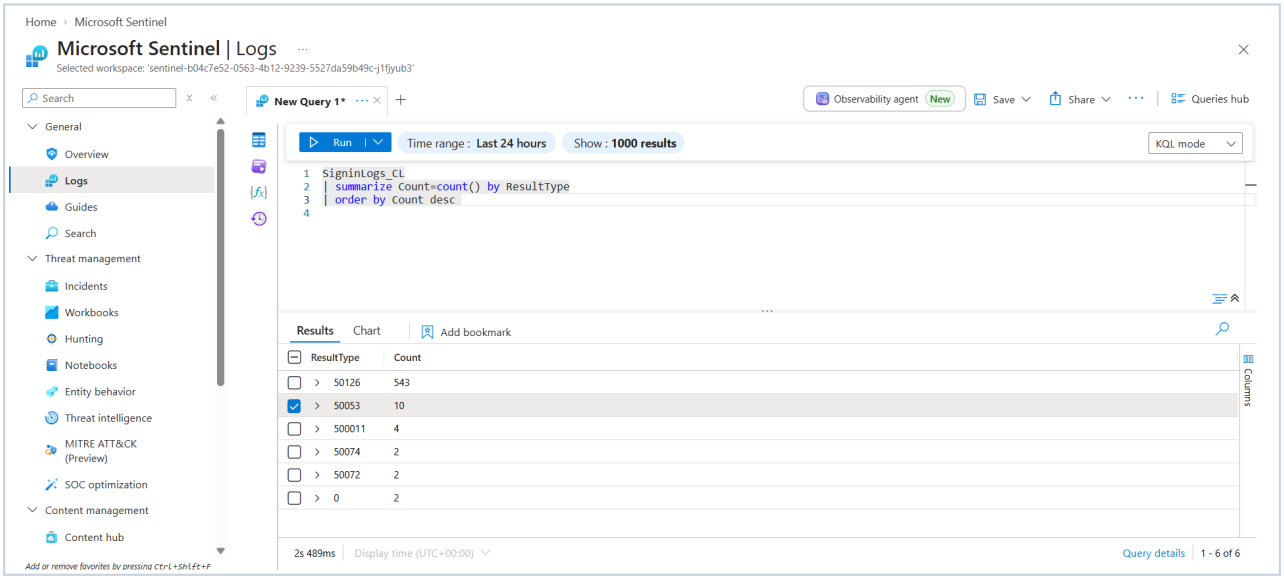


Figure 1. Sign-in events grouped by ResultType, showing one failure code dominating the log

Every following query filters on this dominant failure code so that the analysis stays focused on the spray traffic and is not diluted by normal sign-in noise.

PHASE 2 Attack Source Profiling

With the failure pattern isolated, the next goal was to learn who was behind it. Three attributes were examined: the source IP address, the client user agent, and the geographic location of the traffic.

2.1 Source IP address

Aggregating the failed attempts by `IPAddress` and keeping the top row reveals the most active source. Spray campaigns are often spread over several addresses, so the busiest IP is a starting point for pivoting rather than the whole story. The address resolves to public cloud hosting space, a common choice for attackers who want their traffic to blend in and to be cheap to rotate.

```
SigninLogs_CL
| where ResultType == 50126
| summarize FailedAttempts = count() by IPAddress
| top 1 by FailedAttempts desc
```

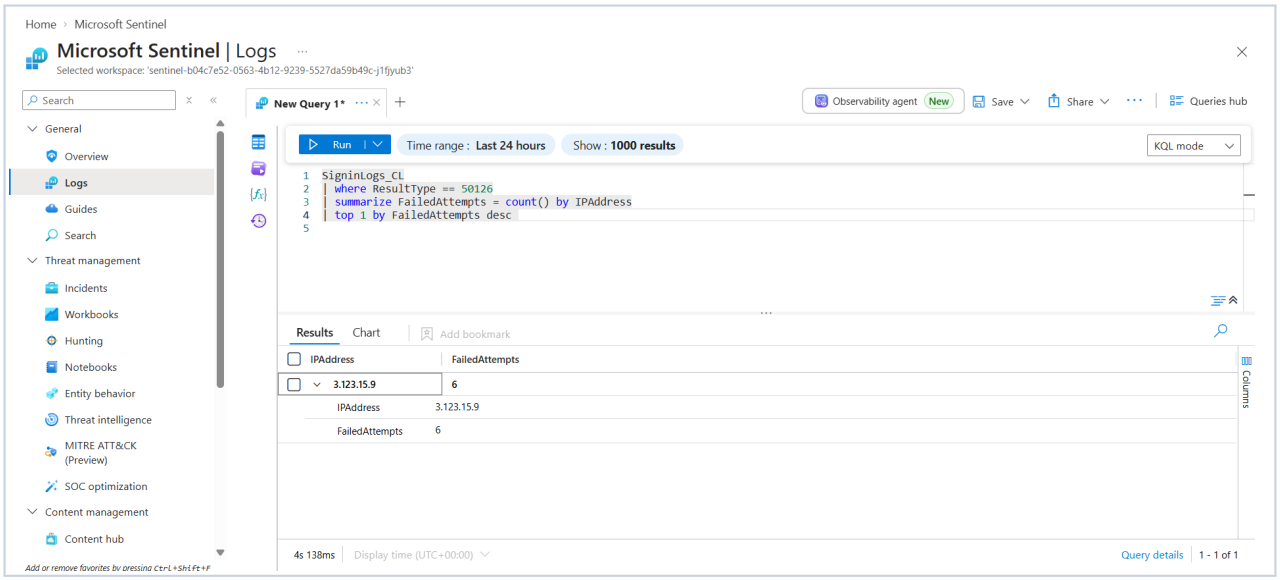


Figure 2. Top source IP address by number of failed attempts

2.2 User agent

The same aggregation by `UserAgent` shows what client the attacker presented. A real user population produces a wide variety of browsers, versions and operating systems. A single static string accounting for all of the failures points to one automation tool configured with a hard-coded header. The version in the string was also noticeably dated, another trait of tooling that is not kept up to date.

```
SigninLogs_CL
| where ResultType == 50126
| summarize FailedAttempts = count() by UserAgent
| top 1 by FailedAttempts desc
```

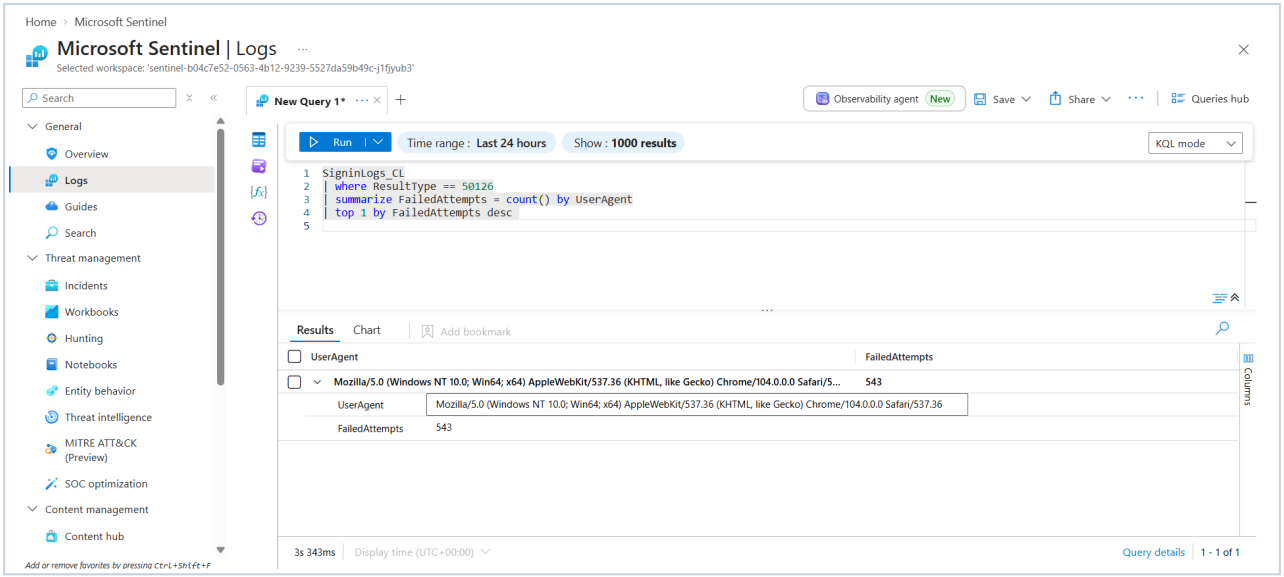


Figure 3. Most common user agent among the failed attempts

2.3 Geography

`LocationDetails` is a nested field holding the city, region, country and coordinates that Entra ID resolved for each sign-in. Grouping the failures by this field shows that the traffic came from one location only. Because attackers routinely use VPNs, proxies or hosted infrastructure, this is treated as the apparent origin and a useful blocking or conditional-access signal, not as definitive attribution.

```
SigninLogs_CL
| where ResultType == 50126
| summarize attemptcount = count() by LocationDetails
| top 1 by attemptcount desc
```

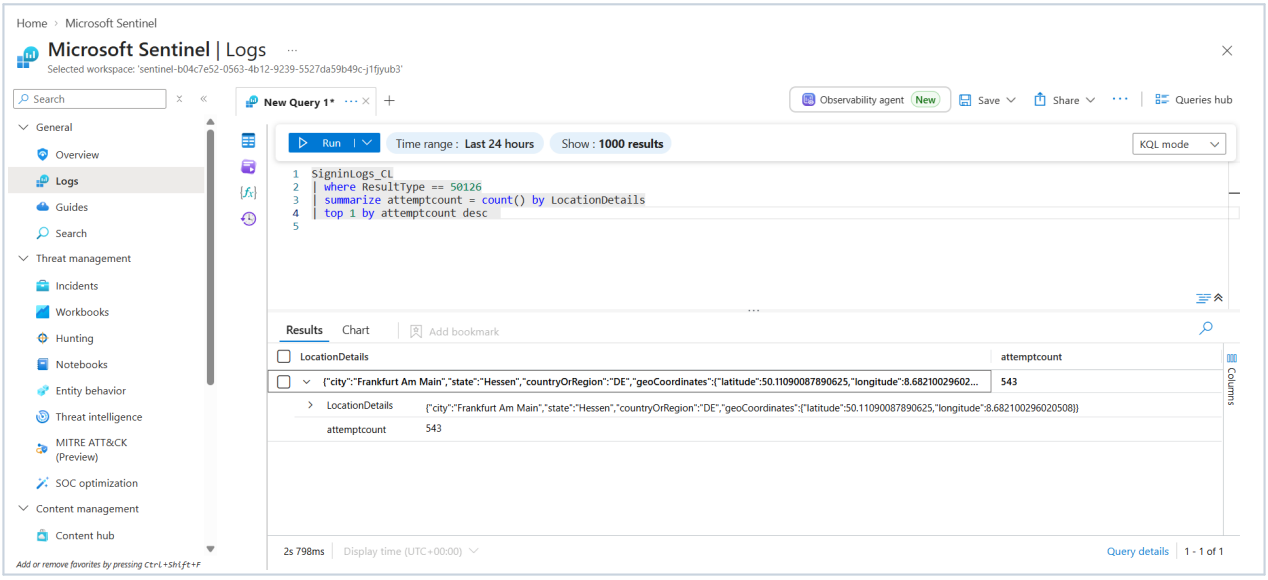


Figure 4. Failed attempts grouped by resolved location details

PHASE 3 Attack Timeline and Scope

3.1 Start of the attack

Taking the minimum creation time across all spray failures marks the earliest observable activity. This anchors the incident timeline, defines the window for all later log analysis, and later lets the response team compare attack start against detection time.

```
SigninLogs_CL
| where ResultType == 50126
| summarize firstFailedAttempt = min(CreatedDateTime)
| project firstFailedAttempt
```

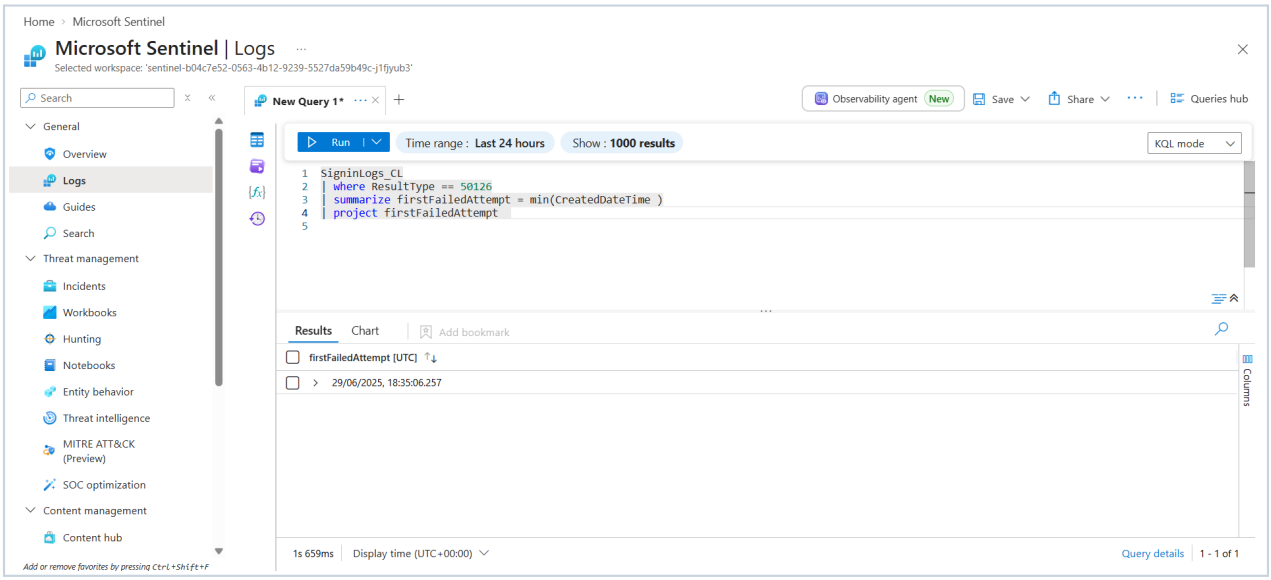


Figure 5. Earliest failed attempt in the log, used as the start of the attack window

3.2 Number of targeted accounts

To measure how wide the spray was, I counted distinct user principal names under the organisation's domain. Restricting to the corporate domain removes any unrelated or malformed names from the count, and `dcount()` collapses the repeated attempts per user into a single entry. A high distinct count together with a low number of attempts per account is the defining shape of a spray, as opposed to a brute force attack against one account.

```
SigninLogs_CL
| where ResultType == 50126
| where UserPrincipalName endswith "@compliantsecure.store"
| summarize UniqueUsers = dcount(UserPrincipalName)
```

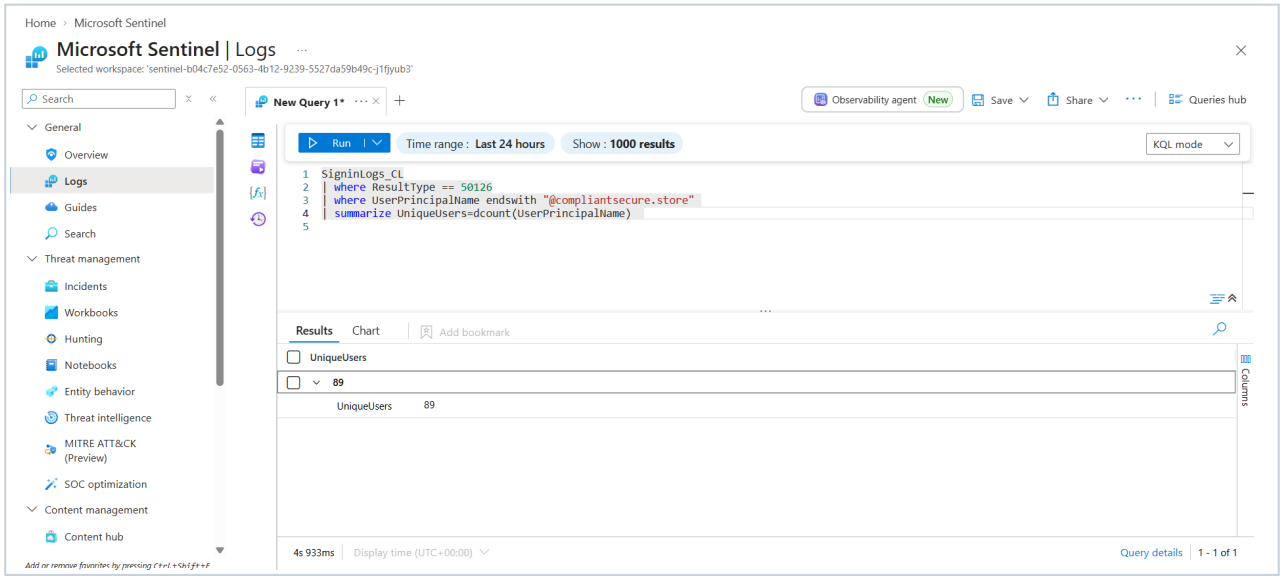


Figure 6. Distinct targeted user principals within the corporate domain

PHASE 4 Detection Engineering

Manual hunting finds the attack after the fact. To catch the next one, the pattern needs to be encoded as a scheduled analytics rule. Microsoft Sentinel ships ready-made templates for exactly this scenario, so the work was to locate the right template, create a rule from it, and adapt it to this environment's custom log schema.

4.1 Locating the template

Searching the rule templates for "spray" returns three password spray detections, one each for AD FS sign-in logs, Entra ID Seamless SSO, and Entra ID applications. The application-focused template matches the data in this environment: it flags an IP address that fails against multiple accounts inside a short time window and enriches the alert with whether the same IP also had successful sign-ins.

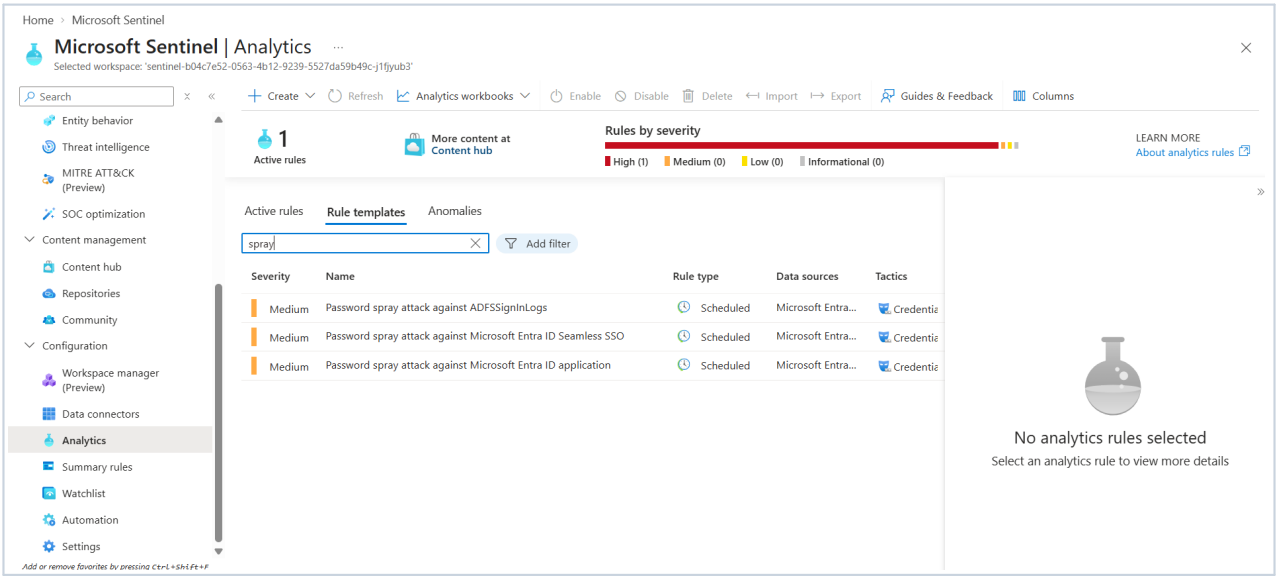


Figure 7. Rule templates filtered by the keyword "spray"

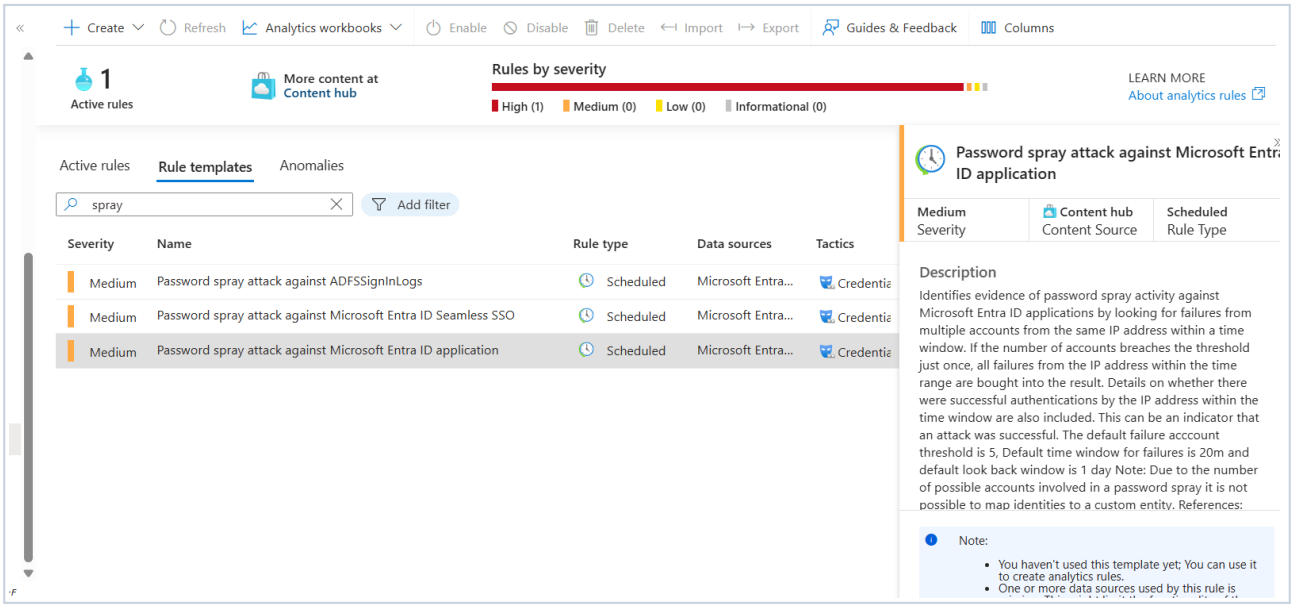


Figure 8. Template details: description, severity, rule type and tactic

4.2 Creating the rule

Selecting *Create rule* opens the analytics rule wizard pre-filled from the template. The General tab carries the name, description, severity and MITRE ATT&CK mapping, all of which were kept, and the work continues on the *Set rule logic* tab.

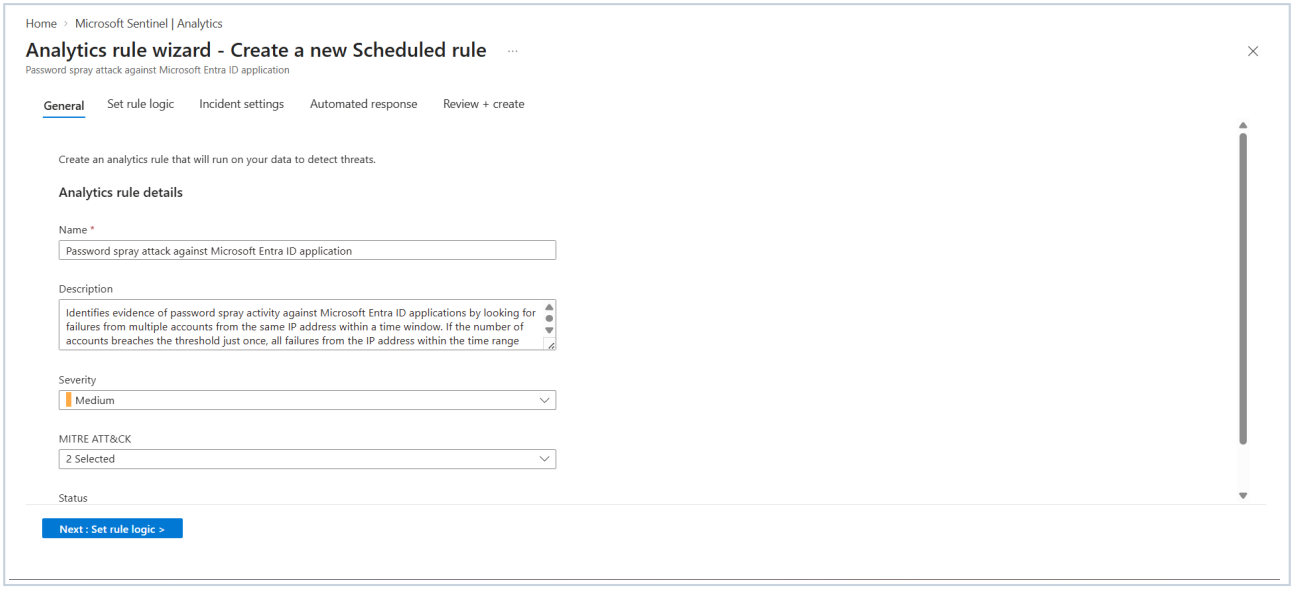


Figure 9. Analytics rule wizard, General tab, pre-populated from the template

4.3 Adapting the query to the custom table

The template is written for the standard Entra ID tables. This environment stores the same data in a custom table, so the unmodified query failed validation: the editor reported that a column referenced by the template could not be resolved. The fix is to make the query schema-tolerant. Instead of referencing fixed column names, the adapted version builds the required fields with `column_ifexists()` wrapped in `extend`, and the main function is pointed at `SigninLogs_CL`. The tunable settings at the top of the query are the look-back period, the authentication window, the failure threshold and the lists of failure and success result codes.

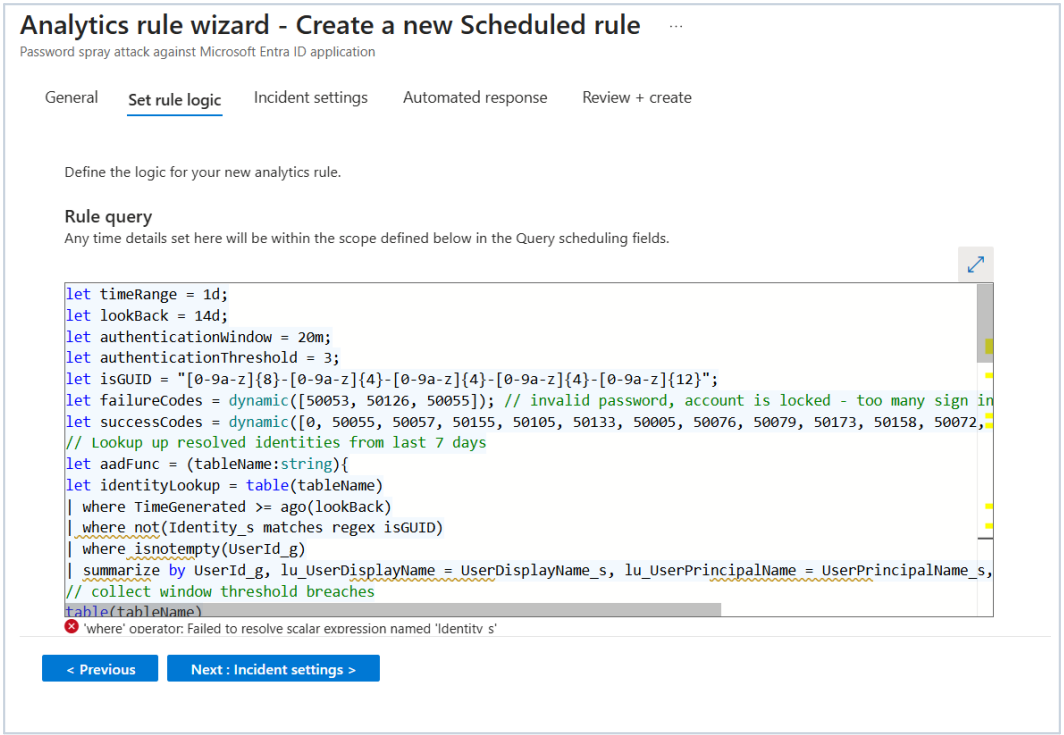


Figure 10. Rule logic tab showing the tunable parameters and the validation error raised by the unmodified template

The following figures walk through the adapted query. The first stage resolves user identities and begins collecting threshold breaches:

```
let timeRange = 1d;
let aadFunc = (tableName:string){
let identityLookup = table(tableName)
| where TimeGenerated >= ago(lookBack)
| extend Identity = tostring(column_ifexists("Identity", ""))
| extend UserId = tostring(column_ifexists("UserId", ""))
| extend UserDisplayName = tostring(column_ifexists("UserDisplayName", ""))
| extend UserPrincipalName = tostring(column_ifexists("UserPrincipalName", ""))
| where not(Identity matches regex isGUID)
| where isnotempty(UserId)
| summarize by UserId, lu_UserDisplayName = UserDisplayName, lu_UserPrincipalName = UserPrincipalName, Type;
// collect window threshold breaches
table(tableName)
| where TimeGenerated > ago(timeRange)
| where ResultType in(failureCodes)
| summarize FailedPrincipalCount = dcount(UserPrincipalName) by bin(TimeGenerated, authenticationWindow), IP
```

View query results >

Figure 11. Identity lookup rebuilt with column_ifexists so it works against the custom schema

Next, failures are binned by time window and counted per IP address, and the breaching IPs are joined back to the full failure data to collect locations and the applications targeted:

```
let timeRange = 1d;
| where FailedPrincipalCount >= authenticationThreshold
| summarize WindowThresholdBreaches = count() by IPAddress, Type
| join kind= inner (
// where we breached a threshold, join the details back on all failure data
table(tableName)
| where TimeGenerated > ago(timeRange)
| where ResultType in(failureCodes)
| extend LocationDetails = todynamic(column_ifexists("LocationDetails", ""))
| extend FullLocation = strcat(LocationDetails.countryOrRegion,'|', LocationDetails.state, '|', LocationDeta
| extend Identity = tostring(column_ifexists("Identity", ""))
| extend UserId = tostring(column_ifexists("UserId", ""))
| summarize StartTime = min(TimeGenerated), EndTime = max(TimeGenerated), make_set(ClientAppUsed,20), make_s
// lookup any unresolved identities
| extend UnresolvedUserId = iff(Identity matches regex isGUID, UserId, "")
| join kind= leftouter (
```

Figure 12. Window threshold breaches summarised per IP and joined back to the failure details

The query then resolves any remaining identities and joins in each IP's success versus failure history, which is what lets the alert show whether a spraying IP also managed to log in:

```
let timeRange = 1d;
| join kind= leftouter (
  identityLookup
) on $left.UnresolvedUserId==$right.UserId
| extend UserDisplayName=iff(isempty(lu_UserDisplayName), UserDisplayName, lu_UserDisplayName)
| extend UserPrincipalName=iff(isempty(lu_UserPrincipalName), UserPrincipalName, lu_UserPrincipalName)
| summarize StartTime = min(StartTime), EndTime = max(EndTime), make_set(UserPrincipalName,20), make_set(Use
| extend FailedPrincipalCount = array_length(set_UserPrincipalName)
) on IPAddress
| project IPAddress, StartTime, EndTime, TargetedApplication=AppDisplayName, FailedPrincipalCount, UserPrinc
| join kind= inner (
table(tableName) // get data on success vs. failure history for each IP
| where TimeGenerated > ago(timeRange)
| where ResultType in(successCodes) or ResultType in(failureCodes) // success or failure types
| summarize GlobalSuccessPrincipalCount = dcountif(UserPrincipalName, (ResultType in (successCodes))), Resul
| where GlobalFailPrincipalCount > GlobalSuccessPrincipalCount // where the number of failed principals is g
```

View query results >

Alert enhancement

Entity mapping

Figure 13. Resolution of unresolved identities and projection of the alert fields

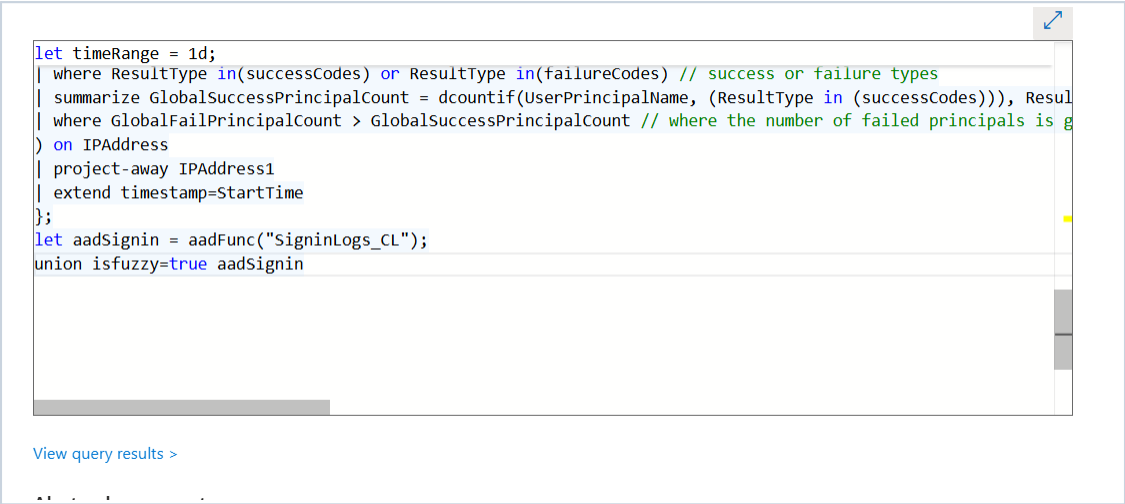


Figure 14. Final stage: filtering to IPs with more failed principals than successful ones, then running the function over the custom table

4.4 Validating the output

Using *View query results* before saving confirms that the rule logic runs and produces rows. Each row is a source IP with the time window of its activity, the application it targeted, the number of distinct accounts it failed against, and the list of those accounts. The rule matches the hunting findings from the earlier phases.

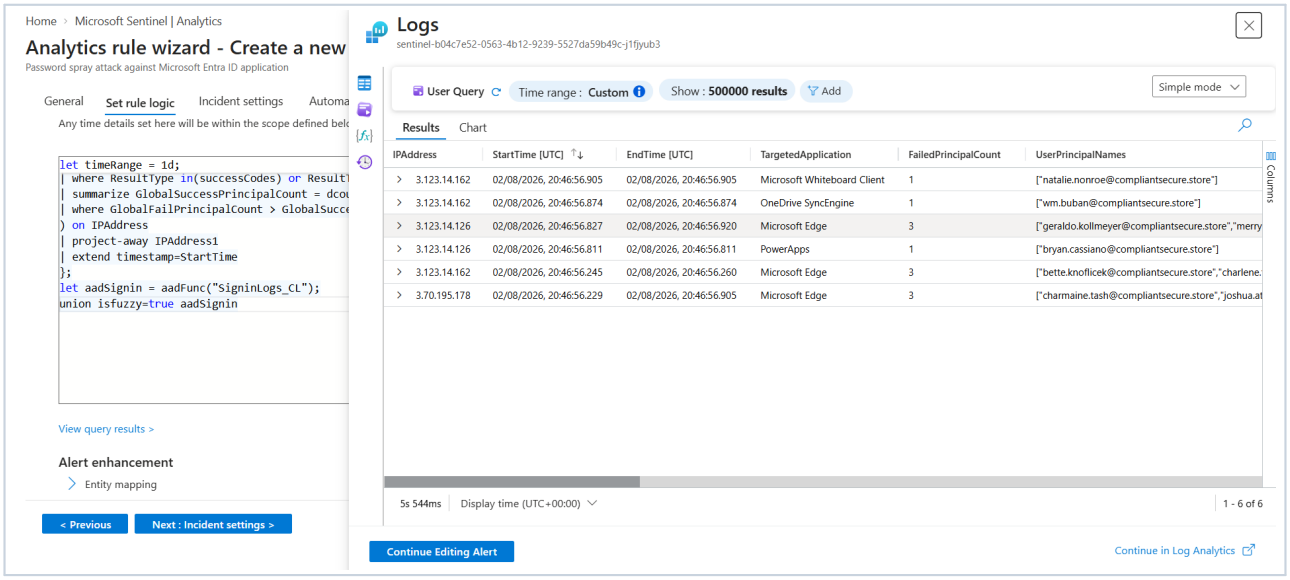


Figure 15. Query results preview inside the rule wizard

4.5 Reviewing the time window

The authentication window controls how failures are grouped when the rule counts distinct accounts per IP. Too small and a slow, patient spray slips between windows; too large and normal activity starts to look suspicious. Reviewing this setting is an important part of tuning the rule for sensitivity versus noise.

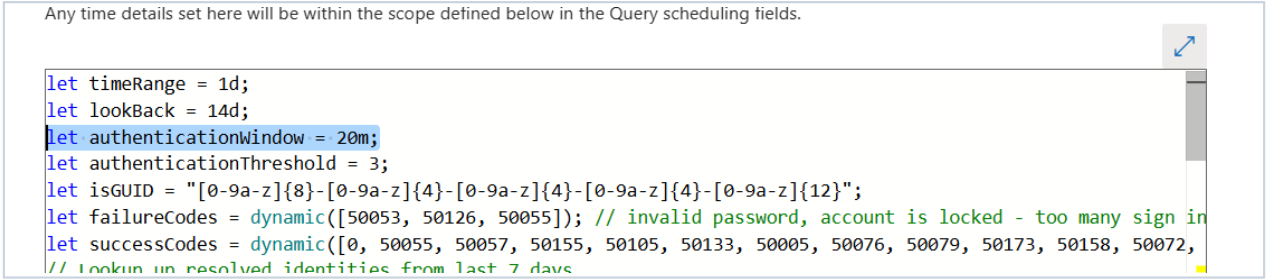


Figure 16. The authentication window parameter in the rule query

PHASE 5 Incident Review

Once the rule was saved and enabled, it ran over the historical data in its look-back period and raised an incident. In the Incidents blade the new entry carries the rule name, medium severity, the Credential Access tactic, and an entities panel listing every attacker IP address correlated by the rule. Seeing all source IPs under one incident, rather than as separate alerts, gives responders the complete picture of a distributed campaign, which is the main operational advantage of a correlation rule over single-IP alerting.

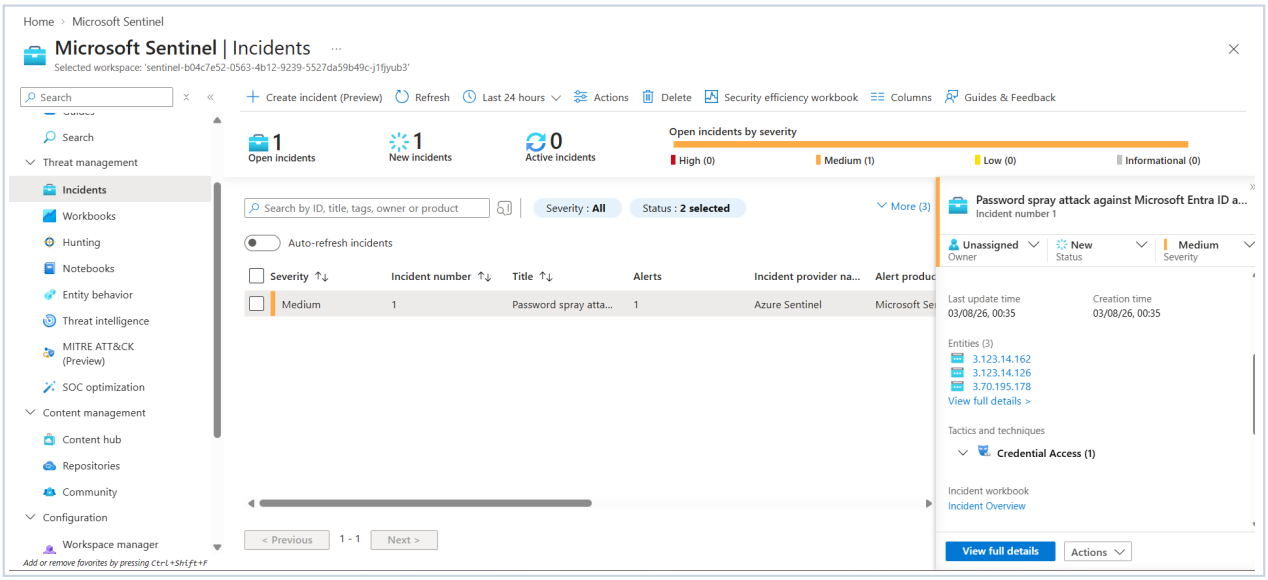


Figure 17. Incident generated by the analytics rule, with attacker IP entities and mapped tactic

PHASE 6 Compromise Identification

The final investigative question was whether the spray succeeded anywhere. Filtering for successful sign-ins (a result type of zero) after the first failed attempt and projecting the time, account and source IP surfaces the only account that authenticated successfully during the campaign. The successful sign-ins originate from an address that was not part of the spray sources, which shows how one weak credential can turn a noisy, mostly failed campaign into real access. This account would be the immediate priority for containment: forced password reset, session revocation and a review of its subsequent activity.

```
SigninLogs_CL
| where ResultType == 0
| where TimeGenerated > datetime(2025-06-29T18:35:00Z)
| project TimeGenerated, UserPrincipalName, IPAddress
```

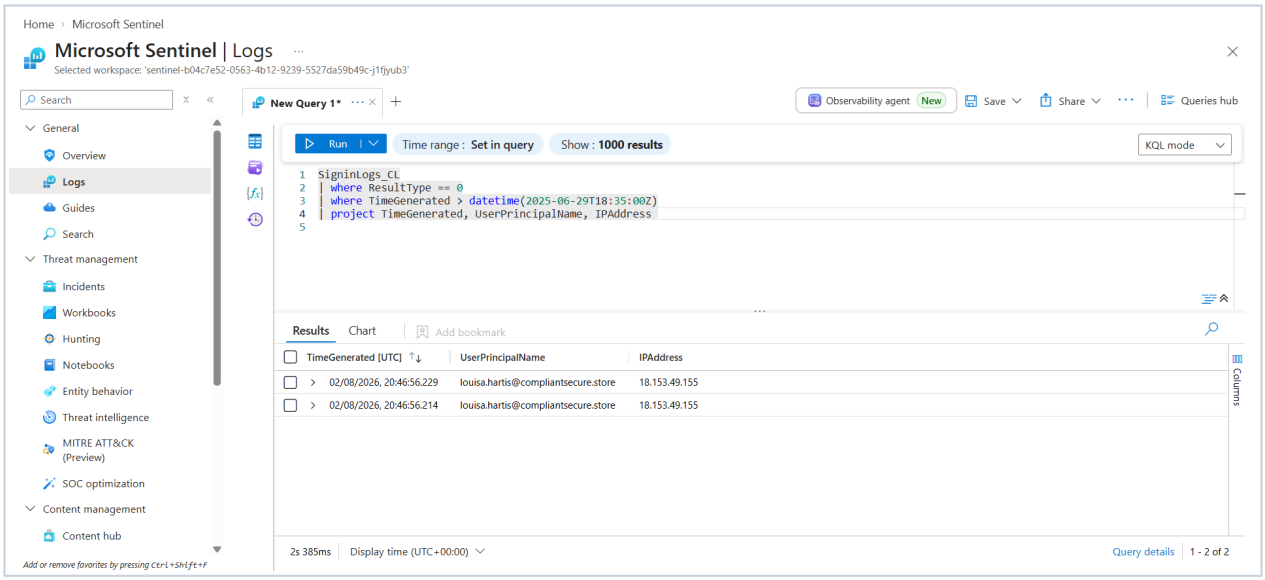


Figure 18. Successful authentications after the start of the attack window

PHASE 7 Mitigation and Hardening

The investigation points to a set of preventive and responsive controls that would reduce both the likelihood and the impact of a similar attack:

- **Block legacy authentication** with a Conditional Access policy. Legacy protocols cannot perform MFA and are the preferred channel for spraying tools, so blocking them removes most of the attack surface.
- **Enforce multi-factor authentication** for all users so that a guessed password alone is not enough to gain access.
- **Tune Smart Lockout** and understand its defaults. It tracks familiar and unfamiliar locations separately, but low-and-slow sprays are designed to stay under its thresholds, so it complements rather than replaces detection.
- **Use Entra Password Protection** with a custom banned password list of organisation-specific terms in addition to the global list, to stop the weak passwords sprays rely on.
- **Protect federated environments** with Extranet Smart Lockout on AD FS, which brings equivalent protection to on-premises federation.
- **Adopt length-focused password guidance** in line with current NIST recommendations, favouring longer passphrases and screening against known-compromised passwords over arbitrary complexity rules.
- **Automate response** with a Sentinel playbook that revokes the sign-in sessions of any account flagged by the rule through the Microsoft Graph API, shrinking the attacker's window of access.
- **Keep the detection tuned**: review the threshold and window values periodically against real traffic to balance sensitivity and false positives.

Key Takeaways

- A password spray is recognised by its shape: many accounts, few attempts each, one dominant failure code, and a uniform client fingerprint.

- Hunting queries and detection rules answer different questions. KQL aggregation explains what happened, while a scheduled analytics rule makes sure it is caught next time.
- Vendor rule templates are a strong starting point but usually need adapting to custom schemas, which is where understanding the query logic matters.
- Correlating several source IPs into a single incident is what turns scattered failed logins into a clear campaign.
- A single weak password is enough for a successful compromise, so prevention (blocking legacy authentication, MFA, banned passwords) matters as much as detection.